

Image Generation

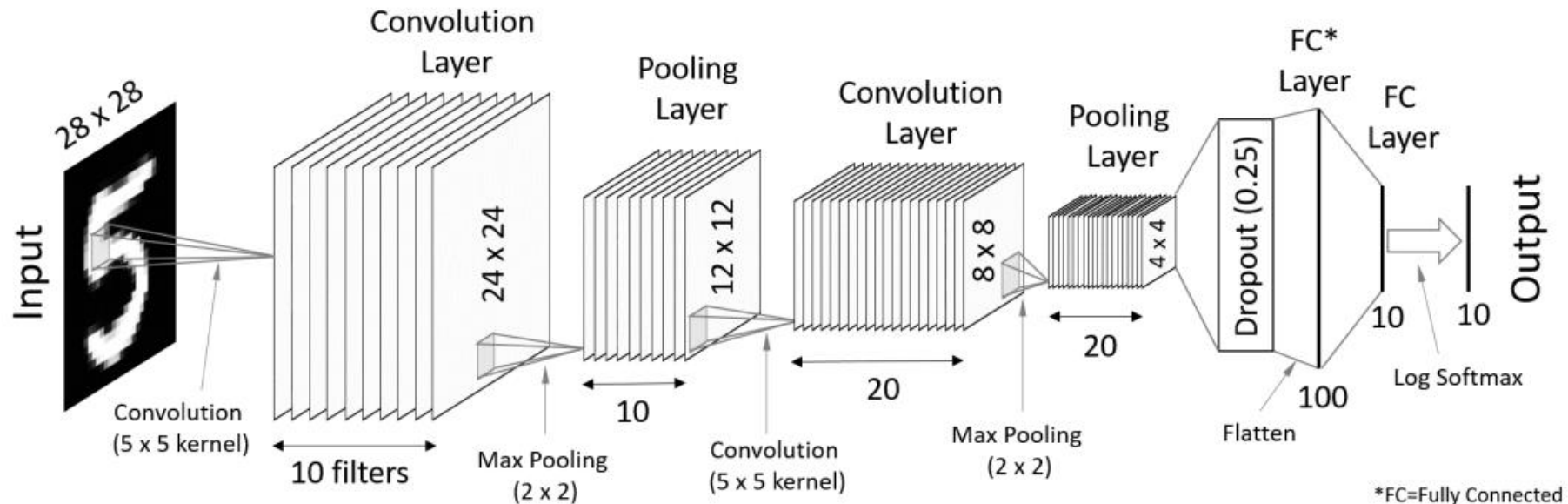
Ronald A. Sumida

rsumida@innovationinsoftware.com



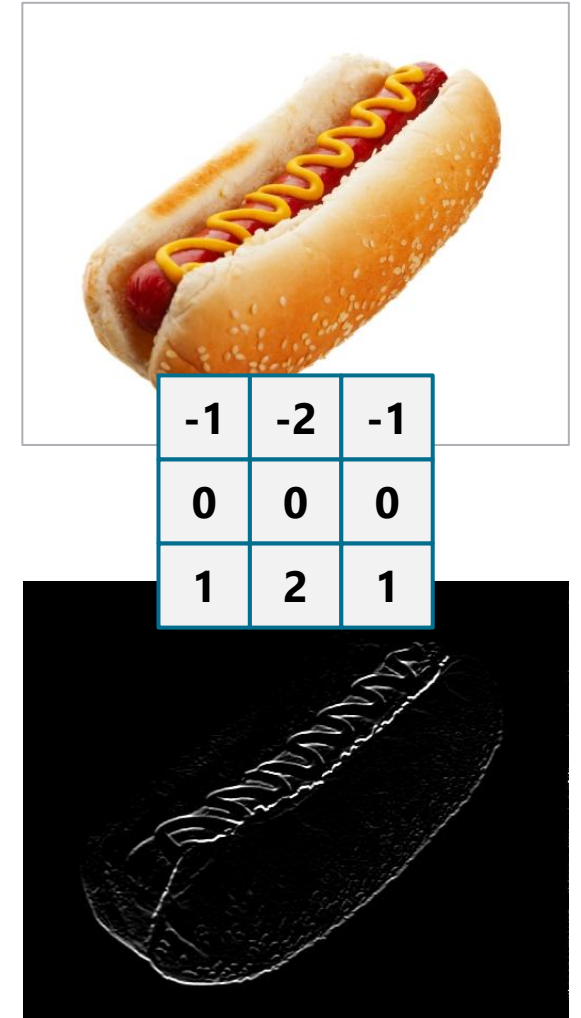
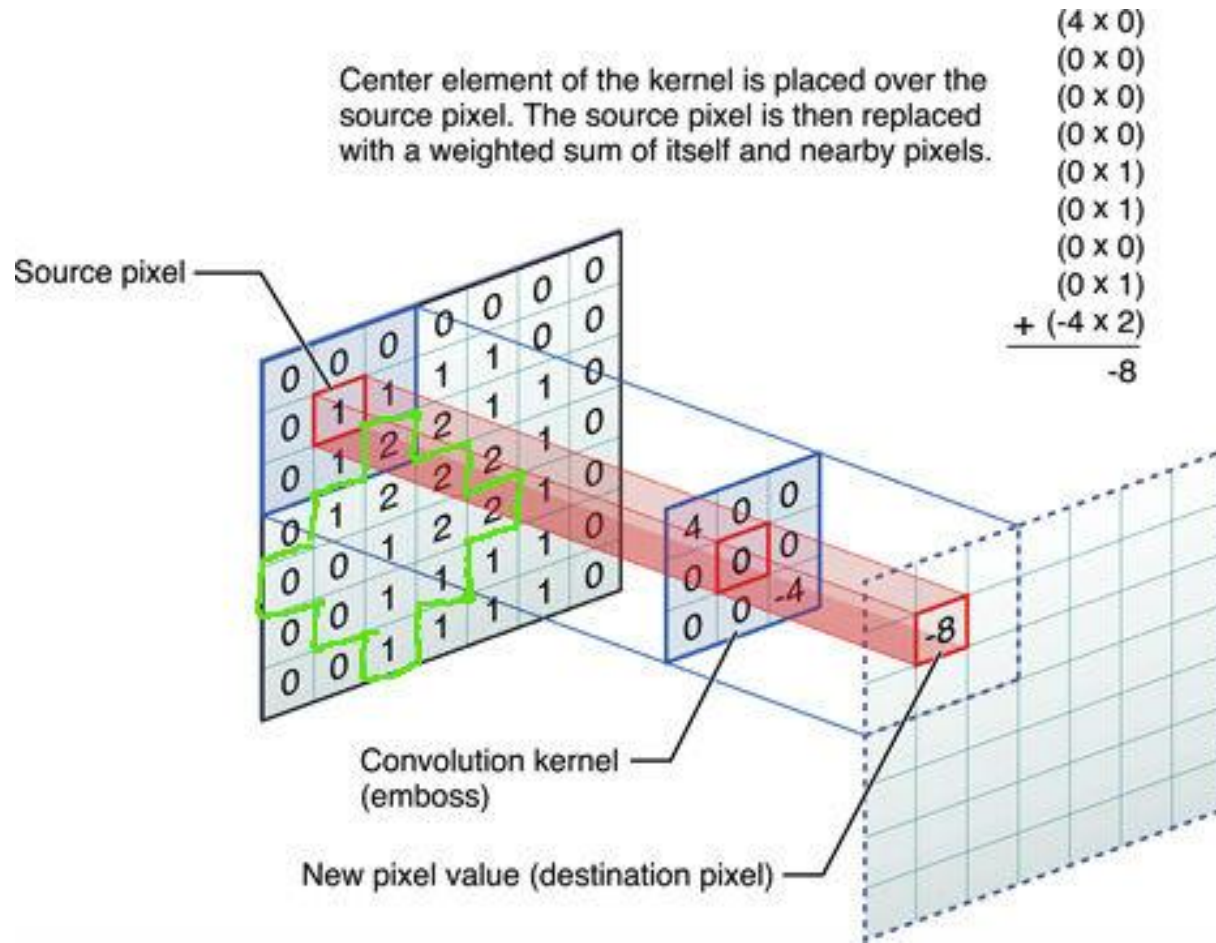
Convolutional Neural Networks (CNNs)

- Excel at computer-vision tasks such as image classification
- Use convolution layers and convolution kernels to create feature maps
- Use pooling layers to subsample feature maps and generalize features



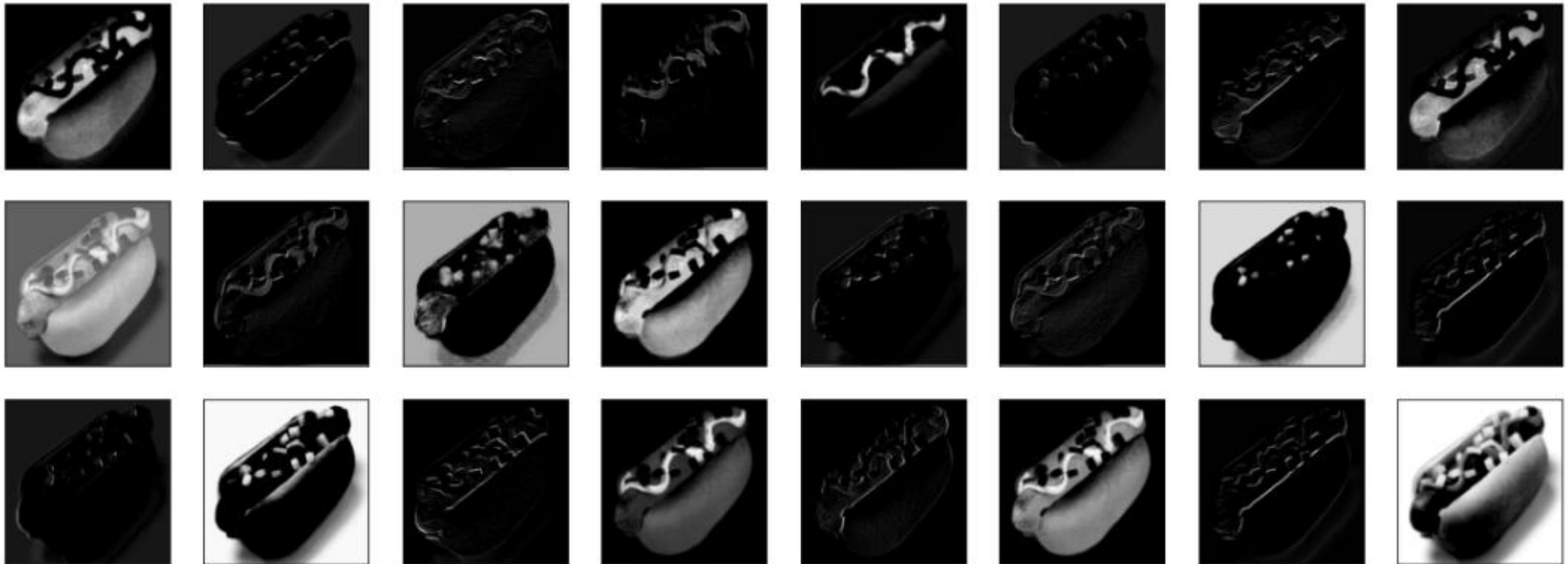
Source: <https://codetolight.wordpress.com/2017/11/29/getting-started-with-pytorch-for-deep-learning-part-3-neural-network-basics/>

Convolution Kernels



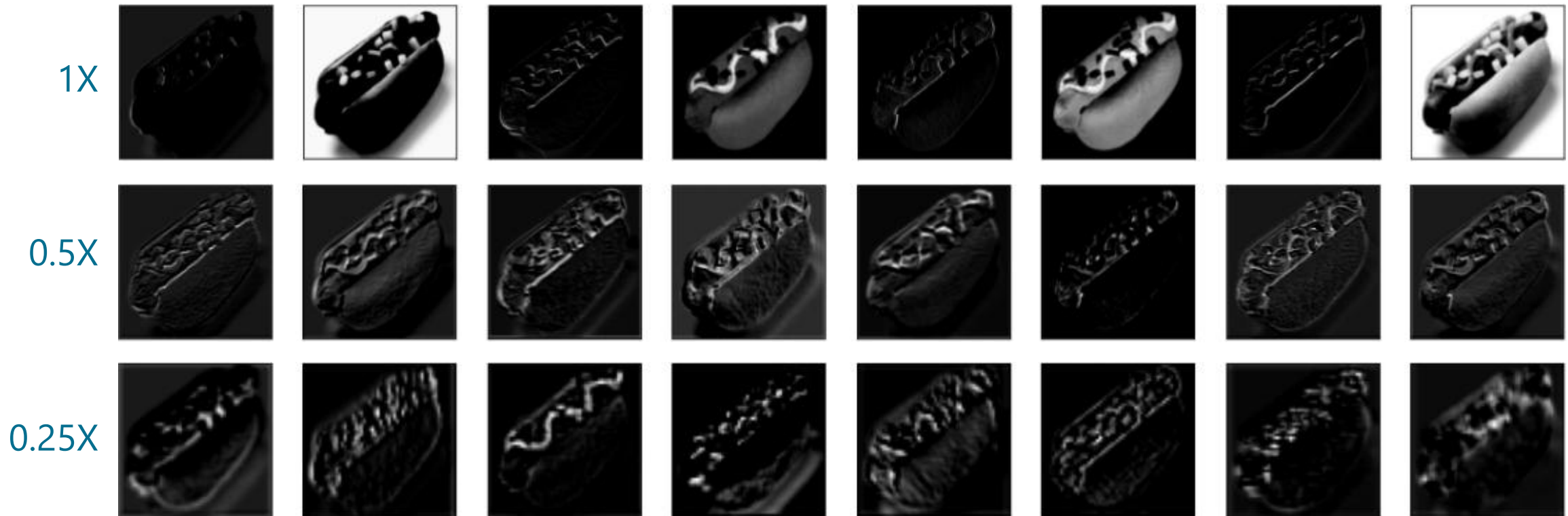
Convolution Layers

- Use convolution kernels to extract features from images
- Use multiple kernels per layer, with values "learned" during training



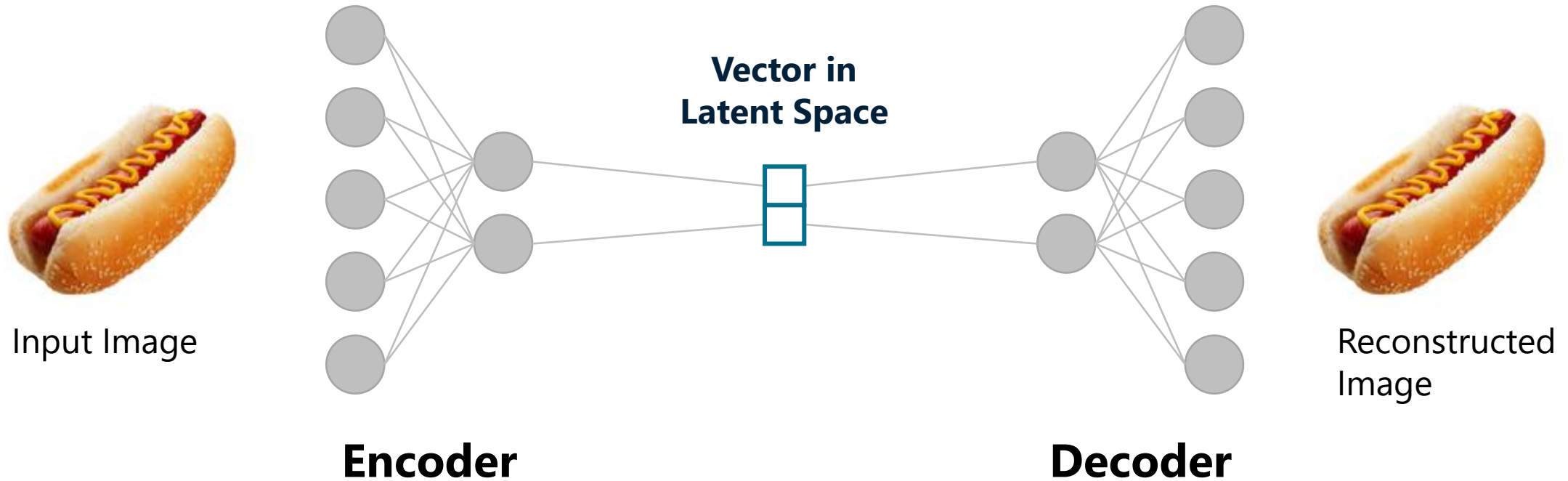
Pooling Layers

- Successively reduce images to half their original size
- Reduce positional sensitivity and extract features at various resolutions



Autoencoders

- Use encoders to reduce inputs to lower-dimensional "latent space" and decoders to reconstruct the original inputs from latent space



Implementing an Autoencoder

```
encoder = Sequential()  
encoder.add(Flatten()) # Reshape 28x28 images to 1D  
encoder.add(Dense(128, activation='relu'))  
encoder.add(Dense(32, activation='relu'))  
  
decoder = Sequential()  
decoder.add(Dense(128, activation='relu'))  
decoder.add(Dense(28 * 28, activation='relu'))  
decoder.add(Reshape((28, 28))) # Reshape 1D array into 28x28 image  
  
model = Sequential([encoder, decoder])  
model.compile(loss='mse', optimizer='adam')  
model.fit(x_train, x_train, epochs=10, validation_data=(x_test, x_test))
```

Implementing a Convolutional Autoencoder

```
encoder = Sequential()
encoder.add(Conv2D(16, (3, 3), activation='relu', strides=2, padding='same',
                  input_shape=(28, 28, 1)))
encoder.add(Conv2D(32, (3, 3), activation='relu', strides=2, padding='same'))
encoder.add(GlobalAveragePooling2D())

decoder = Sequential()
decoder.add(Dense(7 * 7 * 16), activation='relu')
decoder.add(Reshape((7, 7, 16)))
decoder.add(Conv2DTranspose(32, (3, 3), strides=2, activation='relu', padding='same'))
decoder.add(Conv2DTranspose(1, (3, 3), strides=2, activation='relu', padding='same'))
decoder.add(Reshape((28, 28)))

model = Sequential([encoder, decoder])
model.compile(loss='mse', optimizer='adam')
model.fit(x_train, x_train, epochs=10, validation_data=(x_test, x_test))
```

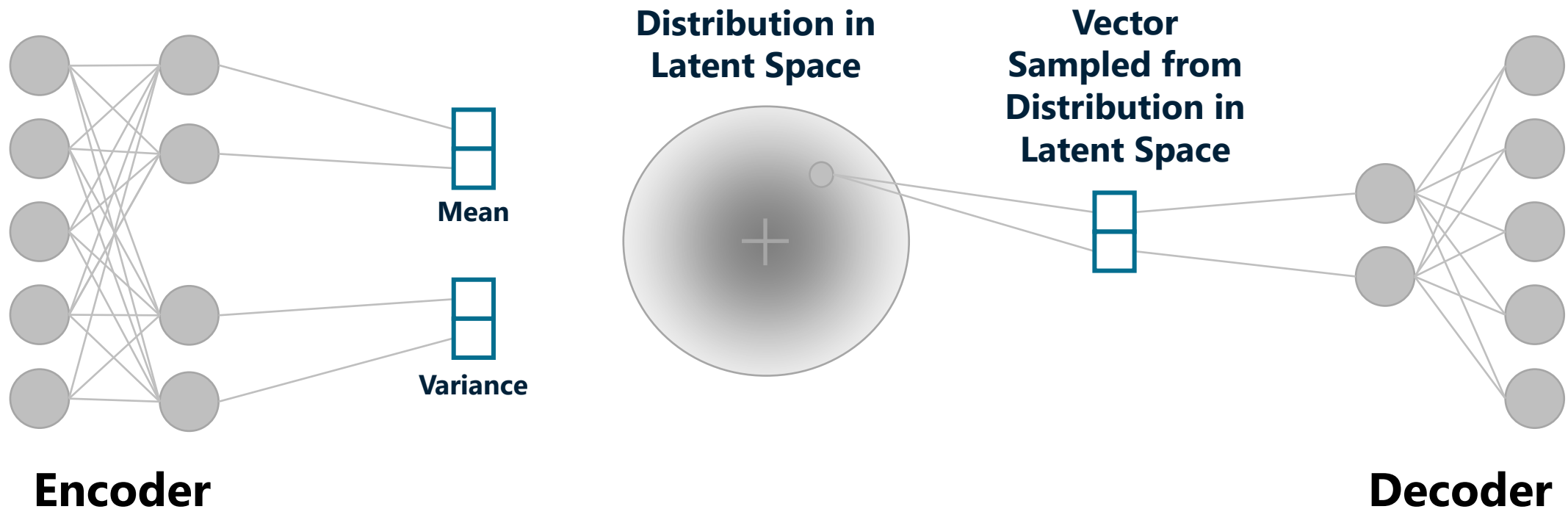

Demo

Autoencoders



Variational Autoencoders (VAEs)

- Provide continuity in latent space by introducing randomness
- Encoder generates probabilistic distributions rather than points



Implementing a Variational Encoder

```
encoder_input = Input(shape=(28, 28))
x = Flatten()(encoder_input)
x = Dense(128, activation='relu')(x)
x = Dense(32, activation='relu')(x)
z_mean = Dense(8)(x) # Outputs vector representing mean in normal distribution
z_log_var = Dense(8)(x) # Outputs vector representing variance in normal distribution

encoder_output = Sampling()([z_mean, z_log_var]) # Custom layer with two inputs that
                                                # outputs vector sampled from normal
                                                # distribution defined by the inputs

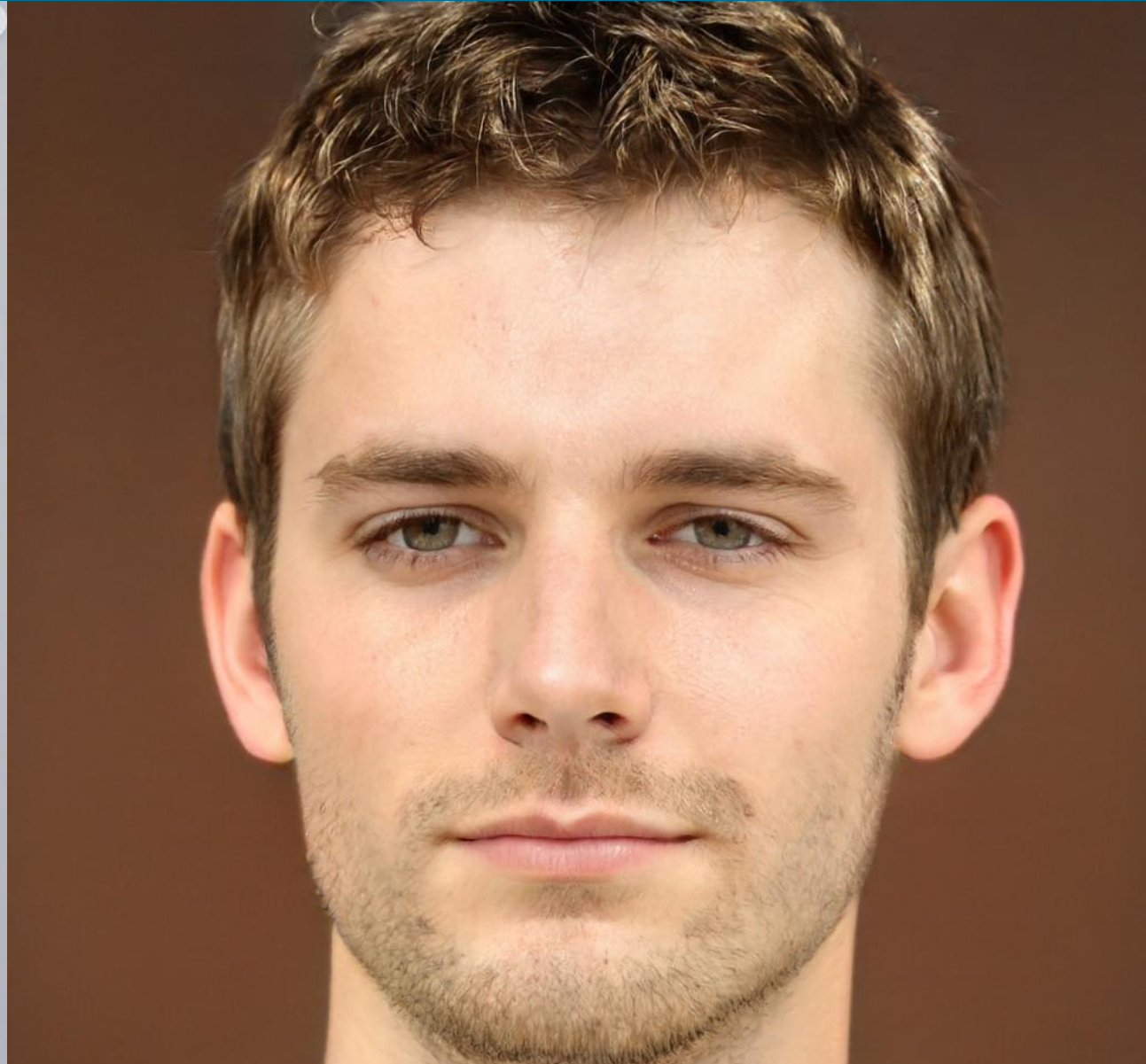
encoder = Model(encoder_input, encoder_output)
```

Demo

Variational Autoencoders

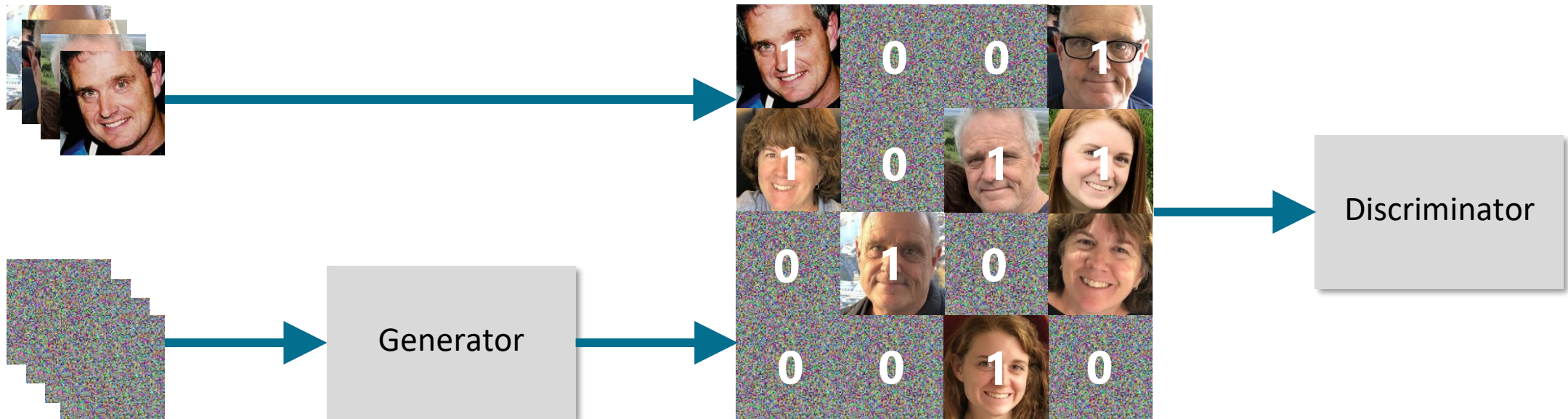


thispersondoesnotexist.com



Generative Adversarial Networks (GANs)

- Use one model (the *discriminator*) to train another model (the *generator*) to produce images from random inputs
- After training, use the generator to produce realistic images



Building and Training GANs

- Architect model to avoid mode collapse and instability
 - Use strided convolutions (strides=2) rather than pooling and upsampling layers in both the generator and the discriminator
 - Use batch normalization after all layers except for the generator's output layer and the discriminator's input layer
 - Use ReLU activation in the generator, except for the last layer, which should use **tanh** activation instead (requires preconditioning of training data)
 - Use leaky ReLU activation in the discriminator
- Implement custom training loop to train generator and discriminator separately in each batch of each epoch

Implementing a Discriminator

```
discriminator = Sequential()  
discriminator.add(Conv2D(16, (3, 3), activation=LeakyReLU(0.2), strides=2,  
                        padding='same'))  
discriminator.add(BatchNormalization()) # Or Dropout  
discriminator.add(Conv2D(32, (3, 3), activation=LeakyReLU(0.2), strides=2,  
                        padding='same'))  
discriminator.add(BatchNormalization()) # Or Dropout  
discriminator.add(Flatten()) # Or GlobalAveragePooling2D  
discriminator.add(Dense(1, activation='sigmoid'))  
discriminator.compile(loss='binary_crossentropy', optimizer='adam')  
discriminator.trainable = False
```

Implementing a Generator

```
generator = Sequential()
generator.add(Dense(7 * 7 * 16))
generator.add(Reshape([7, 7, 16]))
generator.add(BatchNormalization())
generator.add(Conv2DTranspose(32, (3, 3), strides=2, activation='relu', padding='same'))
generator.add(BatchNormalization())
generator.add(Conv2DTranspose(16, (3, 3), strides=2, activation='tanh', padding='same'))

# Form a GAN from the generator and the discriminator
model = Sequential([generator, discriminator])
model.compile(loss='binary_crossentropy', optimizer='adam')

# Don't call fit(); model requires a custom training loop
```

Training the Model

```
for epoch in range(epochs):
    np.random.shuffle(x_train) # Shuffle images at the start of each epoch

    for step in steps_per_epoch:
        # Train the discriminator to differentiate between real and generated images
        # TODO: Fetch a batch of real images and combine with a batch of fake images (x)
        # TODO: Generate labels for images where 0 == fake and 1 == real (y)
        discriminator.train_on_batch(x, y)

        # Train the model (trains the generator but not the discriminator)
        x = np.random.normal(size=(batch_size, input_size))
        y = np.array([1] * batch_size)
        model.train_on_batch(x, y)
```


Demo

Generative Adversarial Networks



Diffusion Models

- Trained to turn random noise into images using diffusion process
- Commercial examples include Stable Diffusion, ImageGen, and DALL·E
- Slower at inference time because decoding (like training) is progressive

Training



During training, images have **random noise** added in stages. The model is trained to **remove the noise** one step at a time, reversing the progression.

Inference (Generation)



At inference time, an image **filled with random pixel values** is input to the model. The model repeatedly denoises the image to produce a final image.

Text-Guided Diffusion



Prompt

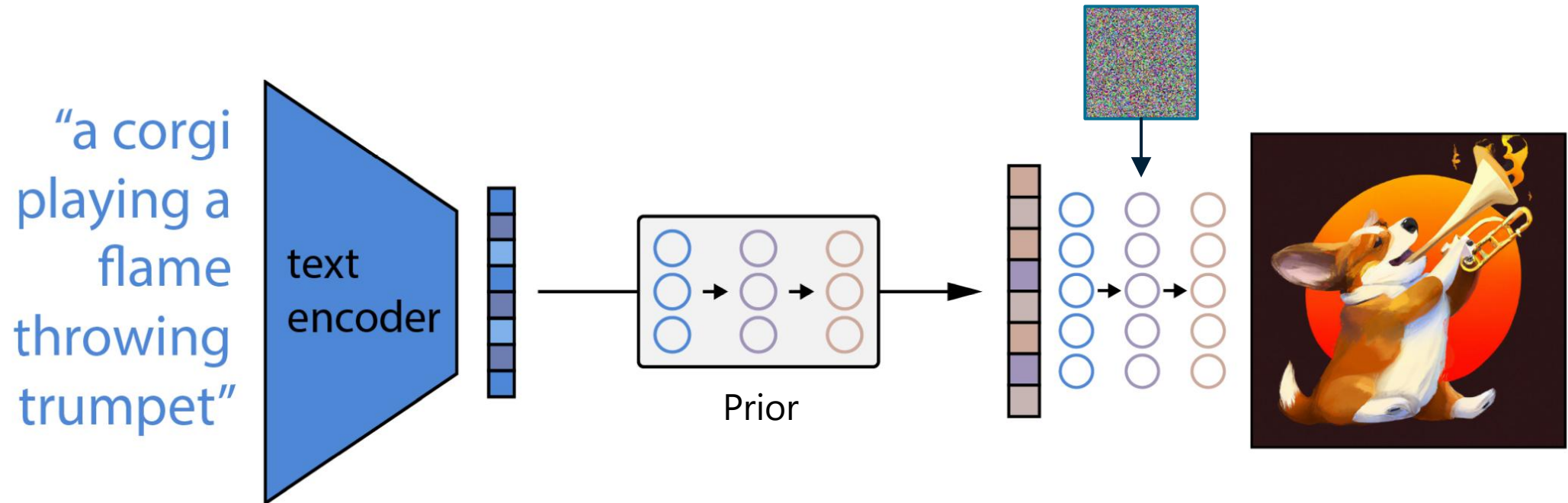
A hyperrealistic photograph of ancient Tokyo/London/Paris architectural ruins in a flooded apocalypse landscape of dead skyscrapers, lens flares, cinematic, hdri, matte painting, concept art, celestial, soft render, highly detailed, cgsociety, octane render, trending on artstation, architectural

DALL·E

- Text-guided diffusion model from OpenAI available by REST API
- DALL·E 2 supports image generation, image variations, inpainting, and outpainting
 - 3.5 billion parameters
 - 650 million text-image pairs
- DALL·E 3 supports image generation with additional features



How DALL·E 2 Works

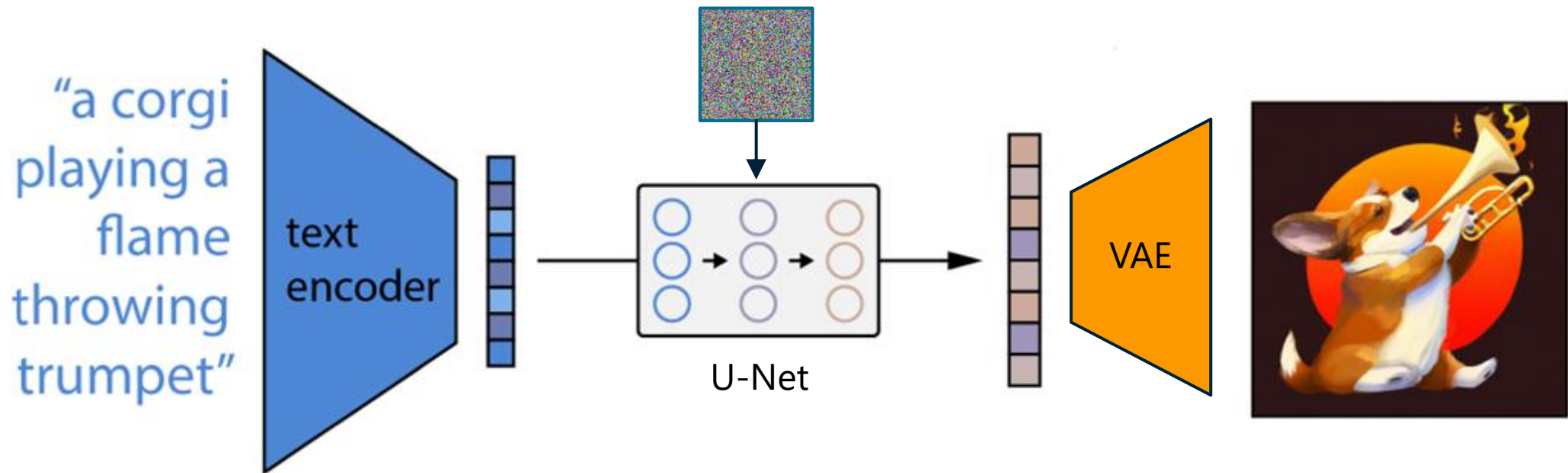


Contrastive Language-Image Pretraining (CLIP) model encodes the prompt, **generating a text embedding** in latent space.

"Prior" model uses reverse diffusion or a transformer to generate an **image embedding** from the text embedding.

Decoder uses the image embedding to **iteratively denoise a random latent image** and generate a **64x64 image** in pixel space. CNNs **upsample the image** to produce the final image.

How Stable Diffusion Works



Contrastive Language-Image Pretraining (CLIP) model based on ClipText (V1) or OpenCLIP (V2) encodes the prompt, **generating a text embedding** in latent space

U-Net uses the text embedding to **iteratively denoise a random latent image**, producing an image embedding. Diffusion is performed in **latent space** that is **48 times smaller** than pixel space.

Variational autoencoder **generates a 64x64 image** from the denoised image embedding produced by the U-Net. Image is **upsampled** to produce the final image.

The Bridge from Pixels to Language

- Traditional generative AI
 - Treated text and pixels as largely separate domains
- DALL·E
 - Important stepping-stone
 - Used language representations to guide a separate diffusion pipeline.
 - Models now deprecated

Multimodal transformers

- More closely integrate language and image generation
- Treat images as a model-readable form of information
- Represent visual information using learned tokens or representations
- Apply transformer-based techniques to visual information
- Parallel the contextual, instruction-driven behavior of LLMs

GPT-Image1

- Released April 2025
- New generation of OpenAI image model
 - Built around a unified multimodal architecture
- Uses the same family of multimodal concepts
 - Already seen in GPT models
- Provides better
 - Instruction following
 - Text rendering
 - Integration with conversational workflows

GPT-Image2

- Released April 2026
- OpenAI's state-of-the-art image model
 - Integrates with reasoning workflows through Thinking Mode
- Supports resolutions up to 2K
- Improvements
 - Layouts, composition, and visual planning
 - Diagrams, infographics, posters, and presentations
 - Typography and multilingual text rendering
 - Consistency across editing and generation tasks

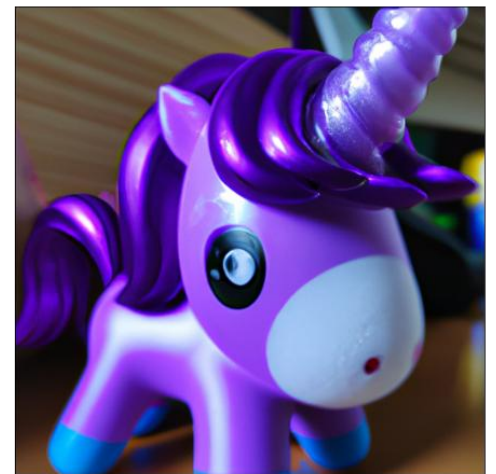
Generating Images with GPT-Image-1

```
import base64
import io
from openai import OpenAI
from PIL import Image

client = OpenAI(api_key='OPENAI_API_KEY')

response = client.images.generate(
    prompt='Photo of a purple unicorn',
    model='gpt-image-1', # or gpt-image-2
    size='1024x1024', # 1024x1536, 1536x1024, or 1024x1024 (default)
    n=1                # Number of images to generate (default == 1)
)

image_data = response.data[0].b64_json
image = Image.open(io.BytesIO(base64.b64decode(image_data)))
display(image)
```



Generating Images with GPT-Image-2

```
client = OpenAI(api_key='OPENAI_API_KEY')

response = client.images.generate(
    prompt='Photo of a purple unicorn',
    model='gpt-image-2',
    size='1024x1024', # 1024x1024, 1024x1536, 1536x1024, 2560x1440, 3840x2160
    quality='auto' # low, medium high and auto
)

image_data = response.data[0].b64_json
image = Image.open(io.BytesIO(base64.b64decode(image_data)))
display(image)
```

Creating Variations of Existing Images

```
response = client.images.edit(  
    model='gpt-image-2',  
    image=open('PATH_TO_IMAGE', 'rb'),  
    prompt='A variation of this image with a more vivid appearance'  
)
```

Original image



Variation created
by DALL·E 2

Inpainting

```
response = client.images.edit(  
    model='gpt-image-2',  
    image=open('PATH_TO_IMAGE', 'rb'), # Path to original image  
    mask=open('PATH_TO_IMAGE_MASK', 'rb'), # Path to same image with transparent pixels  
    prompt='Photograph of two people standing on a cliff overlooking the beach',  
    size='1024x1024'  
)
```

Original Image



Image Mask



Inpainted Image



Outpainting

```
response = client.images.edit(  
    model='gpt-image-2',  
    image=open('PATH_TO_IMAGE', 'rb'),  
    mask=open('PATH_TO_IMAGE', 'rb'), # Same image  
    prompt='Painting of a 17th-century Ditch girl standing in a kitchen',  
    size='1024x1024'  
)
```

Transparent pixels
identifying region
to be outpainted

Region to be
expanded via
outpainting



Outpainted Image



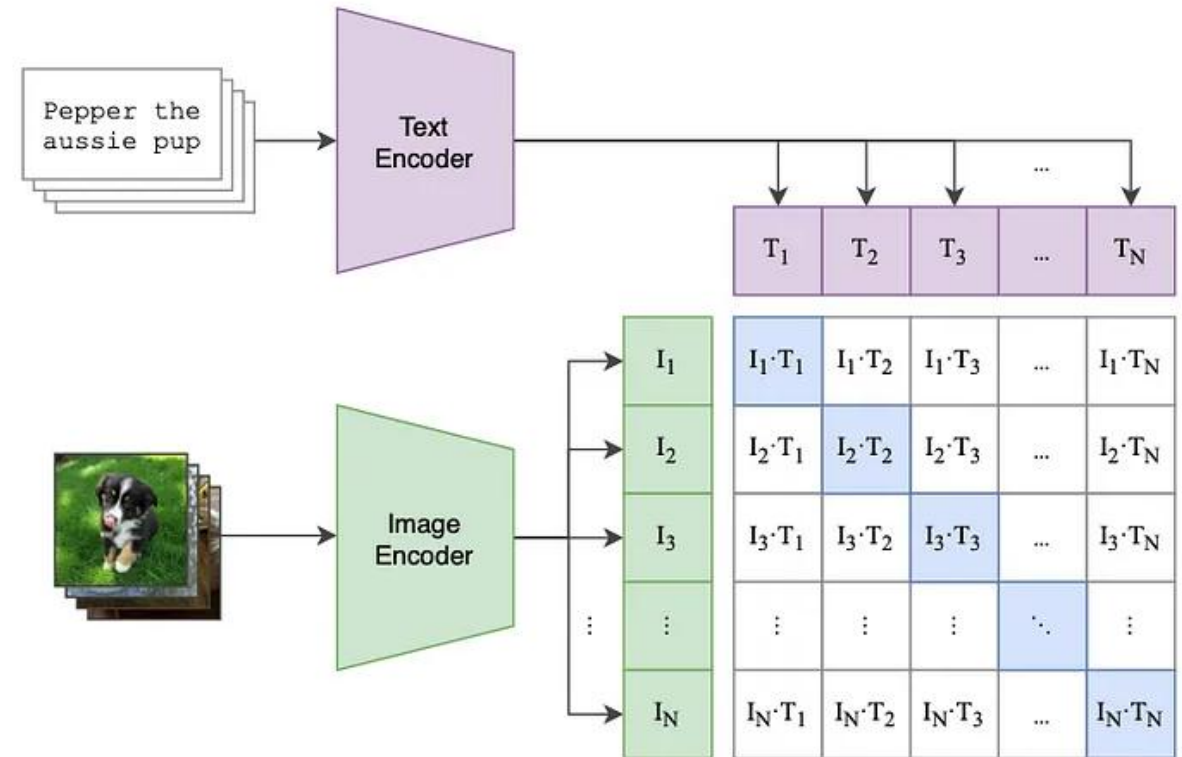
Demo

GPT-Image-1 & 2



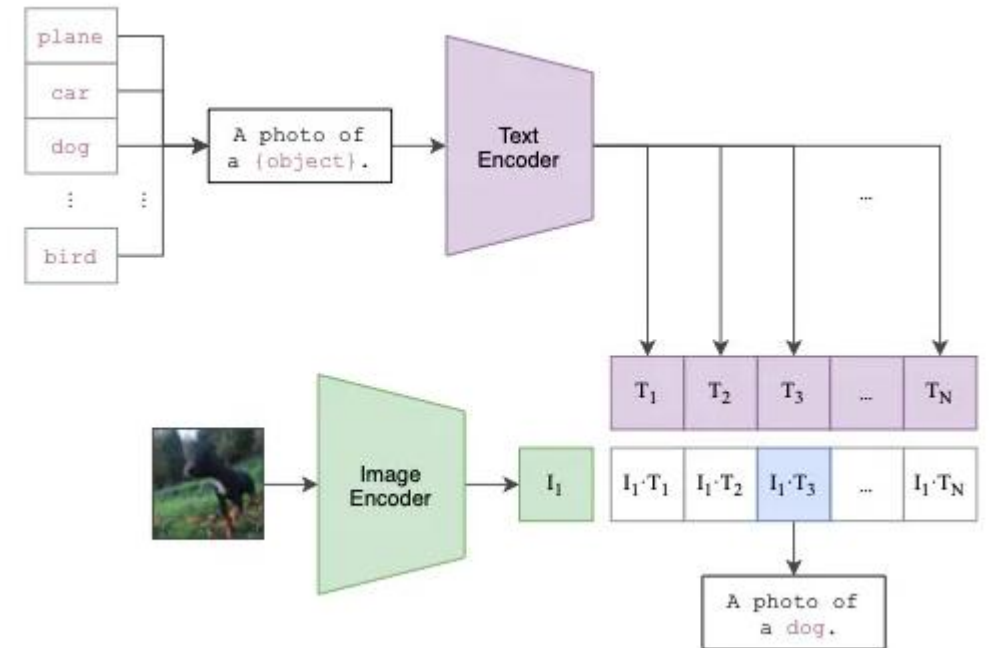
Contrastive Language-Image Pretraining (CLIP)

- Introduced in 2021 by OpenAI and trained on more than **400 million** text-image pairs
- Correlates text and images by training a model to maximize embedding similarities
- DALL·E 2 used CLIP's text encoder to generate embeddings from text prompts



Zero-Shot Image Classification

- Classify photos without:
 - Training a CNN from scratch or using transfer learning with a pretrained CNN
 - Assembling a labeled dataset
- Present model with an image and several possible descriptions
- Model predicts which description is "correct" by computing similarity of image embedding and each text embedding



OpenCLIP

- Open-source version of CLIP
 - Hosted at https://github.com/mlfoundations/open_clip
- OpenAI shared model weights but not the training dataset
- LAION (Large-scale Artificial Intelligence Network) assembled massive text-image datasets, trained several versions of OpenCLIP with them, and published the datasets and model weights
 - Includes LAION-5B dataset with **5.85 billion** text-image pairs
- Many OpenCLIP models trained on these datasets available in Hugging Face's **transformers** package

Using clip-vit-large-patch14

```
from PIL import Image
from transformers import pipeline

model = pipeline(
    model='openai/clip-vit-large-patch14',
    task='zero-shot-image-classification'
)

image = Image.open('PATH_TO_IMAGE')
model(image, candidate_labels=['owl', 'giraffe', 'camel'])
```



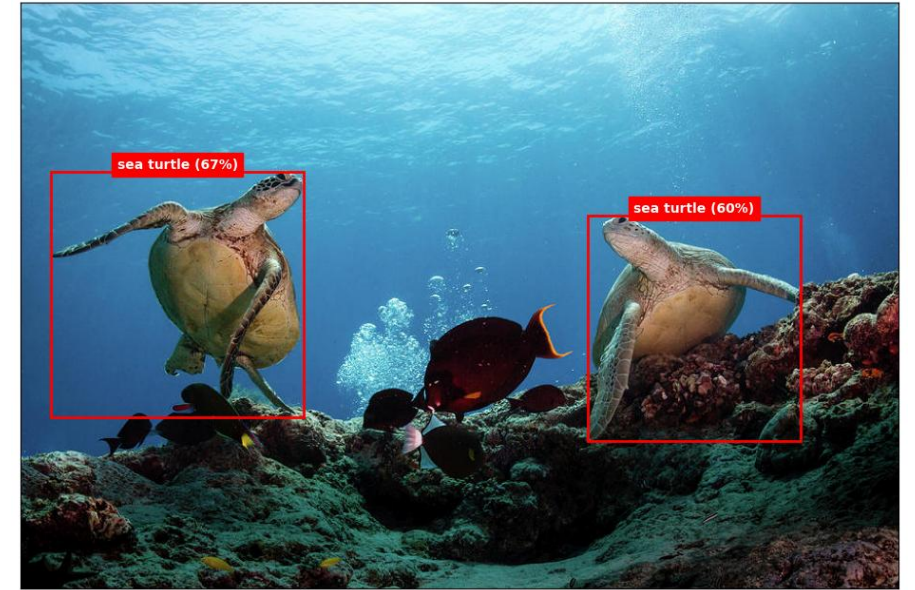
```
[{'score': 0.9978225231170654, 'label': 'giraffe'},
 {'score': 0.002148183062672615, 'label': 'camel'},
 {'score': 2.921208033512812e-05, 'label': 'owl'}]
```

Zero-Shot Object Detection

```
from PIL import Image
from transformers import pipeline

model = pipeline(
    model='google/owlv2-base-patch16-ensemble',
    task='zero-shot-object-detection'
)

image = Image.open('PATH_TO_IMAGE')
candidate_labels = ['parrot', 'sea turtle', 'giraffe']
predictions = model(image, candidate_labels)
```



```
[{'score': 0.6651442646980286,
  'label': 'sea turtle',
  'box': {'xmin': 31, 'ymin': 115, 'xmax': 290, 'ymax': 283}},
 {'score': 0.6040006279945374,
  'label': 'sea turtle',
  'box': {'xmin': 581, 'ymin': 145, 'xmax': 799, 'ymax': 299}}]
```

Demo

Zero-Shot Image Classification

